

Computer Vision and Multiview Geometry

DT084A Lab - 5 Report

Student: Kaan Tekin Öztekin

Date of submission: 07.03.2026

Algorithm	Principle	When Applicable	Pros	Cons
Lucas-Kanade Optical Flow	Tracks motion of feature points assuming brightness constancy	Small motion between frames and feature tracking	Fast and efficient for small motion	Sensitive to illumination change and large motion
Farneback Optical Flow	Computes dense optical flow for all pixels	When full motion field estimation is required	Provides dense motion information	Computationally heavier and sensitive to noise
Mean Shift	Moves search window to region with highest colour density	Objects with distinctive colour distributions	Simple and computationally efficient	Cannot handle scale changes well
CamShift	Adaptive Mean Shift with dynamic window size	Objects that change scale or orientation	Handles scale changes better than Mean Shift	Sensitive to background colours
Template Matching	Searches for the best match of a template in the next frame	Object appearance remains constant	Simple implementation	Sensitive to scale, rotation and lighting
Kalman Filter	Predicts object state using a motion model	Smooth motion with predictable dynamics	Handles noise and allows prediction	Requires predefined motion model
Particle Filter	Tracks objects using probabilistic particles	Non-linear motion or uncertainty	Robust to ambiguity and non-linear motion	Computationally expensive

Table 1: Traditional tracking algorithms in computer vision

Difference Between Object and Feature Tracking

Tracking objects based on colours and tracking image features represent two different approaches in computer vision.

Object tracking methods follow an entire region based on its appearance, usually using colour distributions or intensity patterns. Algorithms such as Mean Shift and CamShift use colour histograms to locate the object in the next frame.

- + Works well when objects have distinctive colours
- + Simple and computationally efficient
- Sensitive to illumination changes
- May fail if the background has similar colours

Feature tracking follows distinctive points such as corners or keypoints across frames. Methods like Lucas–Kanade optical flow track these points using image gradients and motion estimation.

- + More robust to illumination changes
- + Useful for motion estimation and geometric analysis
- Requires textured regions with detectable features
- Feature points may disappear due to motion blur or occlusion

Optical Flow Computation using Lucas–Kanade

The Lucas–Kanade optical flow method estimates the motion of pixels between two consecutive frames by assuming that the motion is constant within a small neighbourhood. In this problem, we consider a 3×3 neighbourhood and compute the optical flow for the central pixel.

Neighbourhood at time t

The 3×3 patch extracted from the frame at time t is

$$\begin{bmatrix} 1 & 2 & 2 \\ 7 & 5 & 5 \\ 5 & 4 & 4 \end{bmatrix}$$

Neighbourhood at time $t + 1$

The corresponding neighbourhood in the next frame is

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 7 & 5 \end{bmatrix}$$

From visual inspection of the two frames, the same intensity pattern appears shifted one pixel to the right and one pixel downward between the frames.

The Lucas–Kanade method is based on the optical flow constraint equation:

$$I_x u + I_y v + I_t = 0$$

where

- I_x and I_y are the spatial image gradients,
- I_t is the temporal gradient between frames,
- u and v represent the horizontal and vertical motion components.

For a single pixel this equation provides one constraint with two unknowns. Therefore, Lucas–Kanade assumes that all pixels inside the 3×3 neighbourhood share the same motion vector.

Lucas–Kanade System of Equations

For each pixel in the neighbourhood we obtain an equation of the form

$$I_x^{(i)} u + I_y^{(i)} v = -I_t^{(i)}$$

Stacking the equations for all pixels yields the system

$$A \begin{bmatrix} u \\ v \end{bmatrix} = b$$

with

$$A = \begin{bmatrix} I_x^{(1)} & I_y^{(1)} \\ I_x^{(2)} & I_y^{(2)} \\ \vdots & \vdots \\ I_x^{(9)} & I_y^{(9)} \end{bmatrix}, \quad b = \begin{bmatrix} -I_t^{(1)} \\ -I_t^{(2)} \\ \vdots \\ -I_t^{(9)} \end{bmatrix}$$

The motion vector is obtained using the least squares solution

$$\begin{bmatrix} u \\ v \end{bmatrix} = (A^T A)^{-1} A^T b$$

Estimated Motion

From the two frames we observe that the entire 3×3 patch has moved one pixel to the right and one pixel downward between time t and $t + 1$.

Therefore, the optical flow vector for the central pixel is

$$\begin{bmatrix} u \\ v \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$$

which means that the point moves one pixel in the horizontal direction and one pixel in the vertical direction between the two frames.

Task 3: Lucas–Kanade Optical Flow on Two Images

The implementation was carried out in Python using OpenCV. First, the two images were read from the `input/` directory and converted to grayscale. Then, good feature points were detected in the first image using the Shi–Tomasi corner detector through the `cv2.goodFeaturesToTrack()` function. These detected points were used as the initial tracking locations.

After feature detection, Lucas–Kanade optical flow was computed with `cv2.calcOpticalFlowPyrLK()`. The resulting motion vectors were visualized by drawing lines from the original feature positions in the first image to their estimated positions in the second image.

Fig 1 shows the comparison of three different Lucas–Kanade settings applied to the full scene. The first result corresponds to a low number of iterations without pyramid levels. The second result uses a higher number of iterations, again without pyramids. The third result uses both a high number of iterations and a pyramid-based coarse-to-fine strategy.



Figure 1: Comparison of Lucas–Kanade optical flow results for the full scene: low iterations without pyramid, high iterations without pyramid, and high iterations with pyramid-based coarse-to-fine processing.

In the first configuration, where only a few iterations were used and no image pyramid was applied, the motion vectors were limited and many tracked points remained very short or weak. This indicates that the optimization stopped early and could not refine the feature locations sufficiently. As a result, the estimated motion remained less informative.

In the second configuration, where the number of iterations was increased, the optical flow result became slightly more stable. More features were successfully refined, and the motion vectors became somewhat clearer. However, since the estimation was still performed only at a single image scale, the algorithm remained limited when the displacement between the two images became too large.

The most distinct behaviour appeared in the third configuration, where the pyramid-based coarse-to-fine strategy was enabled. In this case, many more and longer motion vectors became visible. This is because the image pyramid allows the algorithm to first estimate motion on lower-resolution versions of the images, where large displacements appear smaller and are easier to match. The estimated motion is then refined progressively at higher resolutions. Therefore, coarse-to-fine processing improves the ability of Lucas–Kanade to deal with larger inter-frame motion.

To better illustrate this effect, a zoomed-in view of the central region containing the marker pen and the eraser is shown in Figure 2. In this enlarged comparison, the difference between the three settings becomes easier to observe.



Figure 2: Zoomed comparison of Lucas–Kanade optical flow results in the marker and eraser region for the three parameter settings.

In the zoomed region, the low-iteration result contains only a limited number of short displacement vectors, while the high-iteration result refines slightly more correspondences. The pyramid-based result produces significantly more visible motion vectors, especially around the vertical marker and the edges of the eraser. This shows that coarse-to-fine estimation is more capable of handling larger or more complex motion patterns.

At the same time, the pyramid-based result also contains some long or irregular vectors. These can be interpreted as mismatches caused by the difficulty of the scene transformation. Since the second image is not only shifted but also rotated, different parts of the image move in different directions. Lucas–Kanade assumes local brightness constancy and relatively small motion inside each neighbourhood. When this assumption is partially violated by rotation or larger displacement, some incorrect correspondences may appear.

Overall, the experiment shows that increasing the number of iterations improves local refinement, while the coarse-to-fine strategy makes Lucas–Kanade more robust to larger motion between frames. However, when the transformation includes rotation, the optical

flow field becomes more complex, and some motion vectors may become inaccurate. Thus, the pyramid-based approach provides the strongest response among the tested settings, but it may also introduce more visible mismatches in difficult regions.

References

- [1] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2022.
- [2] OpenCV Documentation, <https://docs.opencv.org/>
- [3] B. Lucas and T. Kanade, “An Iterative Image Registration Technique with an Application to Stereo Vision,” *Proceedings of the Imaging Understanding Workshop*, 1981.
- [4] D. Comaniciu, V. Ramesh, and P. Meer, “Kernel-Based Object Tracking,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2003.